순천향대학교

ARTIFICIAL INTELLIGENCE CONVERGENCE SOFTWARE
순천향대학교 인공지능 융합 소프트웨어 연구실

CNN, VGGNet, Receptive Field, 고찰

발표 |박성진
010 – 9450 – 6778
ppssjj020222@gmail.com
순천향대학교 멀티미디어관

CONTENTS

01 CNN

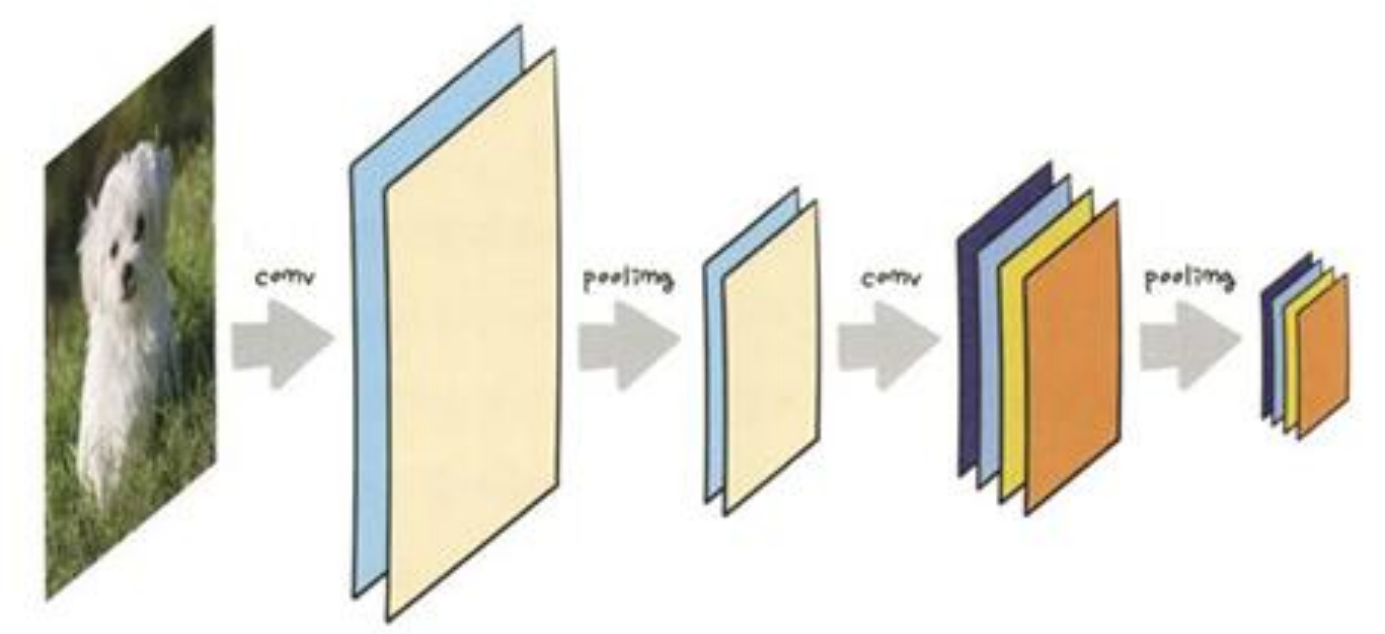
02 VGGNet

03 Receptive Field

04 고찰

01 | CNN

특징 추출부터 분류



- Conv : 이미지에서 특징 추출
  - 선, 모서리, 질감, 특정 패턴
- Pooling : 정보 요약, 압축
  - 계산량 감소, 과적합 완화

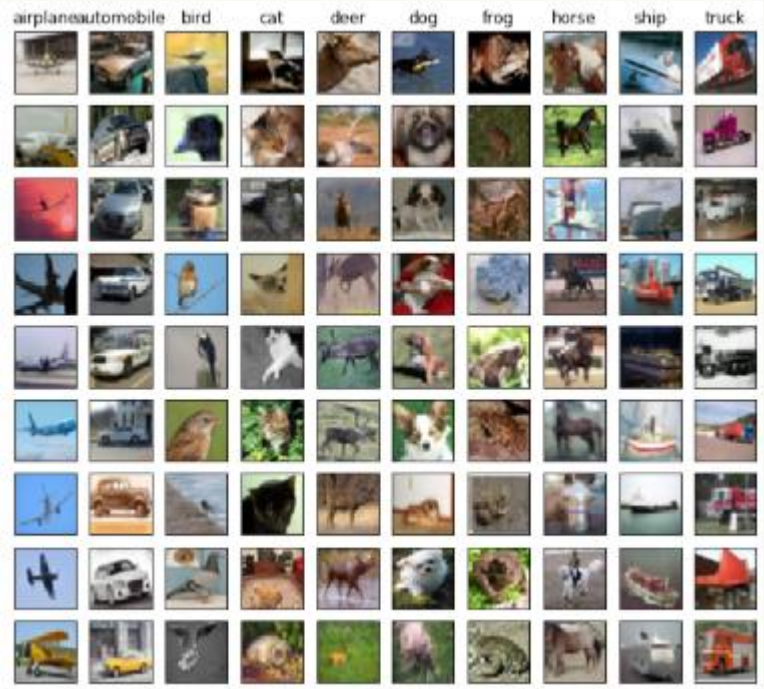
- 강아지 사진이라면

01 | CNN

특징 맵 실험 결과 분석

- CNN 실제로 어떤 특징을 포착 하는지 실험
  - 컨벌루션 레이어를 거치며 특징맵이 어떻게 변화하는지

- CIFAR-10 데이터셋
  - 강아지, 고양이, 비행기 등 10개 클래스
  - Traing: 5만 장, Test: 1만장



01 | CNN

특징 맵 실험 결과 분석

```
class CNN_deep(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv_block1 = nn.Sequential(nn.Conv2d(3,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU(),
                                     nn.Conv2d(32,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU())
    self.Maxpool1 = nn.MaxPool2d(2)

    self.conv_block2 = nn.Sequential(nn.Conv2d(32,64,3,padding=1),
                                     nn.BatchNorm2d(64),
                                     nn.ReLU(),
                                     nn.Conv2d(64,64,3,padding=1),
                                     nn.BatchNorm2d(64),
                                     nn.ReLU(),
                                     nn.Conv2d(64,64,3,padding=1),
                                     nn.BatchNorm2d(64),
                                     nn.ReLU())
    self.Maxpool2 = nn.MaxPool2d(2)

    self.conv_block3 = nn.Sequential(nn.Conv2d(64,128,3,padding=1),
                                     nn.BatchNorm2d(128),
                                     nn.ReLU(),
                                     nn.Conv2d(128,128,3,padding=1),
                                     nn.BatchNorm2d(128),
                                     nn.ReLU(),
                                     nn.Conv2d(128,128,3,padding=1),
                                     nn.BatchNorm2d(128),
                                     nn.ReLU())
    self.Maxpool3 = nn.MaxPool2d(2)

    self.classifier = nn.Sequential(nn.Linear(128*4*4,512),
                                    nn.ReLU(),
                                    nn.Linear(512,10))

  def forward(self, x):
    x = self.conv_block1(x)
    x = self.Maxpool1(x)
    x = self.conv_block2(x)
    x = self.Maxpool2(x)
    x = self.conv_block3(x)
    x = self.Maxpool3(x)
    x = torch.flatten(x, start_dim=1)
    x = self.classifier(x)
    return x
```

- 세 개의 컨벌루션 블록
- 각 블록 뒤에 Max Pooling 레이어
- 마지막에 두 개의 FC 레이어로 구성된 MLP

- Block 1: 2개의 컨벌루션 레이어
- Block 2: 3개의 컨벌루션 레이어
- Block 3: 3개의 컨벌루션 레이어

01 | CNN

특징 맵 실험 결과 분석

```
class CNN_deep(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv_block1 = nn.Sequential(nn.Conv2d(3,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU(),
                                     nn.Conv2d(32,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU())
    self.Maxpool1 = nn.MaxPool2d(2)

    self.conv_block2 = nn.Sequential(nn.Conv2d(32,64,3,padding=1),
                                     nn.BatchNorm2d(64),
                                     nn.ReLU(),
                                     nn.Conv2d(64,64,3,padding=1),
```

- CNN_deep 신경망 클래스 정의
- 필요한 layer를 정의
- nn.Module 초기화

- nn.Conv2d(3, 32, 3, padding = 1)
  - 입력 채널: 3
  - 출력 채널: 32
  - 커널 크기: 3
  - Padding: 1

- Nn.BatchNorm2d(32)
  - 32개 채널 각각에 대한 정규화 수행
    - 학습 안정화, 속도 가속화

01 | CNN

특징 맵 실험 결과 분석

```
class CNN_deep(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv_block1 = nn.Sequential(nn.Conv2d(3,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU(),
                                     nn.Conv2d(32,32,3,padding=1),
                                     nn.BatchNorm2d(32),
                                     nn.ReLU())
    self.Maxpool1 = nn.MaxPool2d(2)

    self.conv_block2 = nn.Sequential(nn.Conv2d(32,64,3,padding=1),
                                     nn.BatchNorm2d(64),
                                     nn.ReLU(),
                                     nn.Conv2d(64,64,3,padding=1),
```

- nn.ReLU()
  - 비선형 활성화 함수(깊은 신경망 시 유리 – 간단한 계산, 기울기 소실)
  - 음수 0으로 만들고 **양수는 그대로 통과**
- 두 번째 Conv2d(32, 32, 3, padding=1)
  - 앞에서 만든 **32개 특징맵 다시 입력**
  - 또 다른 **32개 특징맵으로 변환**
  - 즉, block1안에서 두번의 합성을 수행
- Self.Maxpool1 = nn.MaxPool2d(2)
  - 2x2 최대 풀링
  - 가로 세로 크기를 절반으로 줄임
  - Ex) 32x32 --> 16x16, 채널 수 유지

01 | CNN

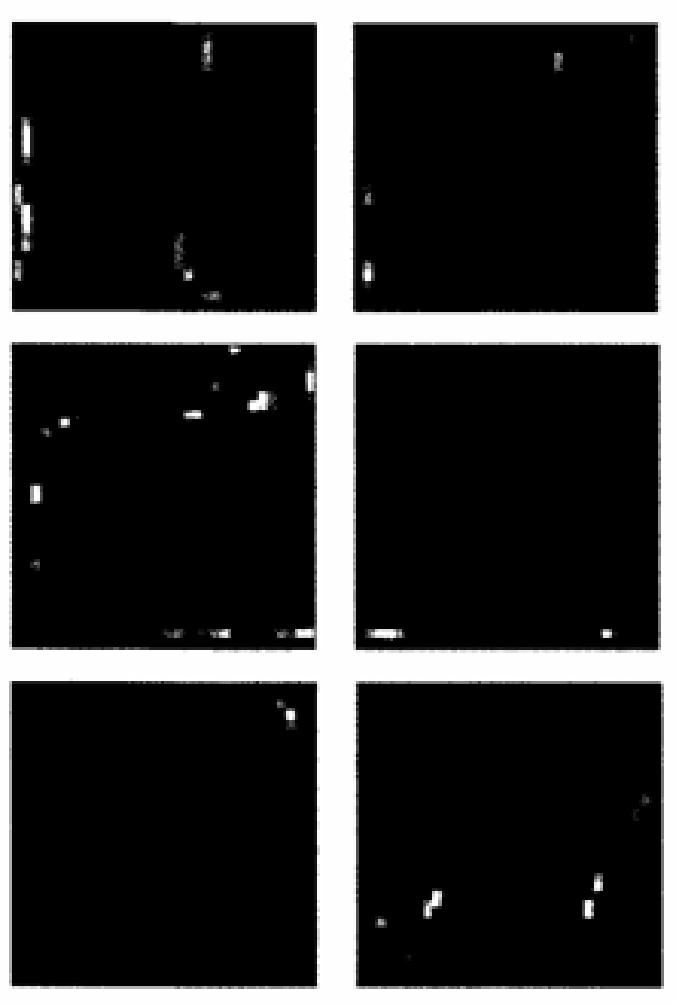
특징 맵 실험 결과 분석

```
                                nn.BatchNorm2d(64),
                                nn.ReLU(),
                                nn.Conv2d(64,64,3,padding=1),
                                nn.BatchNorm2d(64),
                                nn.ReLU())
    self.Maxpool2 = nn.MaxPool2d(2)

    self.conv_block3 = nn.Sequential(nn.Conv2d(64,128,3,padding=1),
                                nn.BatchNorm2d(128),
                                nn.ReLU(),
                                nn.Conv2d(128,128,3,padding=1),
                                nn.BatchNorm2d(128),
                                nn.ReLU(),
                                nn.Conv2d(128,128,3,padding=1),
                                nn.BatchNorm2d(128),
                                nn.ReLU())
    self.Maxpool3 = nn.MaxPool2d(2)

    self.classifier = nn.Sequential(nn.Linear(128*4*4,512),
                            nn.ReLU(),
                            nn.Linear(512,10))

  def forward(self, x):
    x = self.conv_block1(x)
    x = self.Maxpool1(x)
    x = self.conv_block2(x)
    x = self.Maxpool2(x)
    x = self.conv_block3(x)
    x = self.Maxpool3(x)
    x = torch.flatten(x, start_dim=1)
    x = self.classifier(x)
    return x
```

- Block 1: 2개의 컨벌루션 레이어
- Block 2: 3개의 컨벌루션 레이어
- Block 3: 3개의 컨벌루션 레이어
- 블록이 진행 될수록 더 깊은 레이어
  - 더 복잡한 패턴 학습
- nn.Linear(128*4*4, 512)
  - 출력 128*4*4 크기 = 2048
  - 2048차원 벡터를 512차원으로 변환
- nn.ReLU()
  - 비선형성 추가
- nn.Linear(512, 10)
  - 최종 출력 노드 10개 --> 10개 클래스

01 | CNN

특징 맵 실험 결과 분석



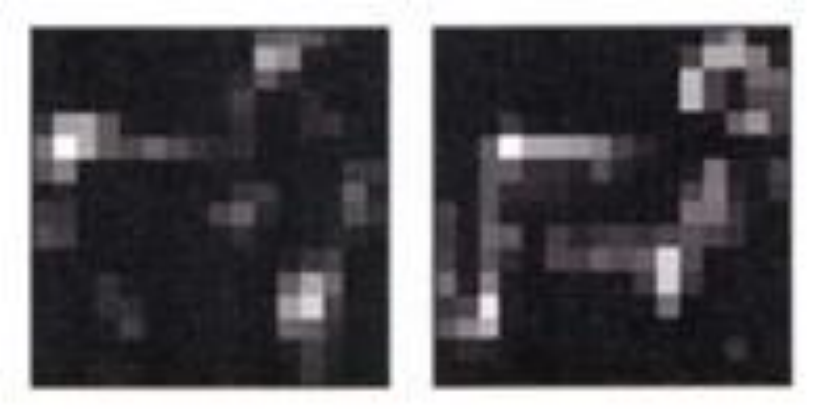
- Conv_block1 : 32개 특징 맵 중 주요한 것

- 첫번 째 줄 : 세로선
- 두번 째 줄 : 가로선
- 세번 째 줄 : 대각선

- 첫 번째 컨벌루션 블록 : “선 특징” 추출
  - CNN의 초기 층에서는 기본적인 형태와 엣지 감지

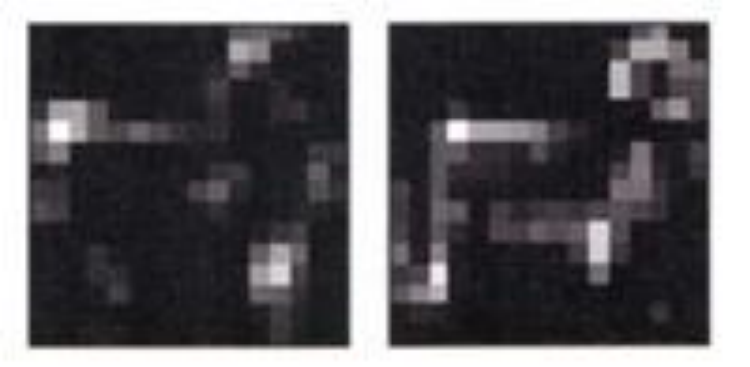
01 | CNN

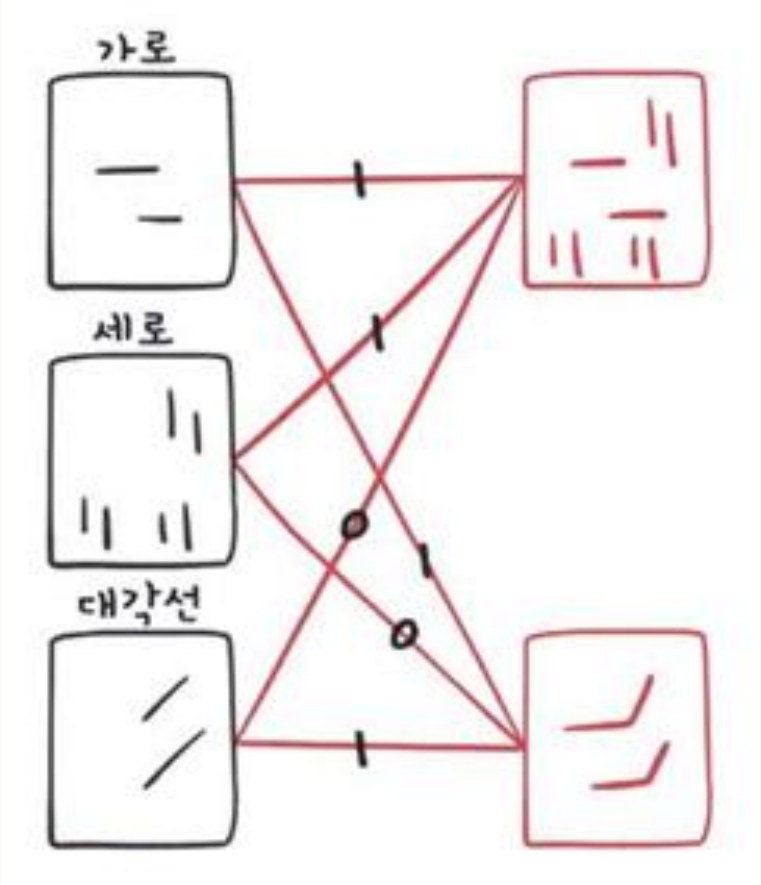
특징 맵 실험 결과 분석



- Conv_block2 : 64개 특징 맵 중 주요한 것
- 이전 블록 단계 특징들이 조합되어 더 복잡한 패턴을 형성

특징 맵 실험 결과 분석





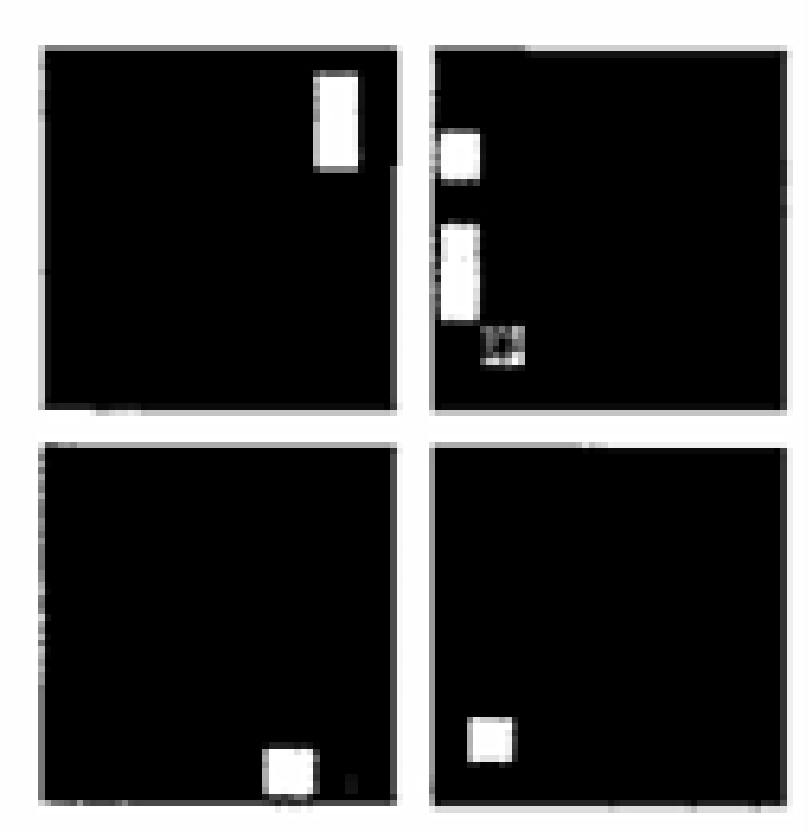
- 가로, 세로 특징 맵(가중치 1), 대각 특징 맵(가중치 0) → 가로, 세로 조합 특징맵 생성

- 가로, 대각 특징 맵(가중치 1), 세로 특징 맵(가중치 0) → 가로, 대각 조합 특징맵 생성

이전 층 의 특징을 다양하게 조합 → 더 복잡하고 추상적인 패턴 포착 가능

01 | CNN

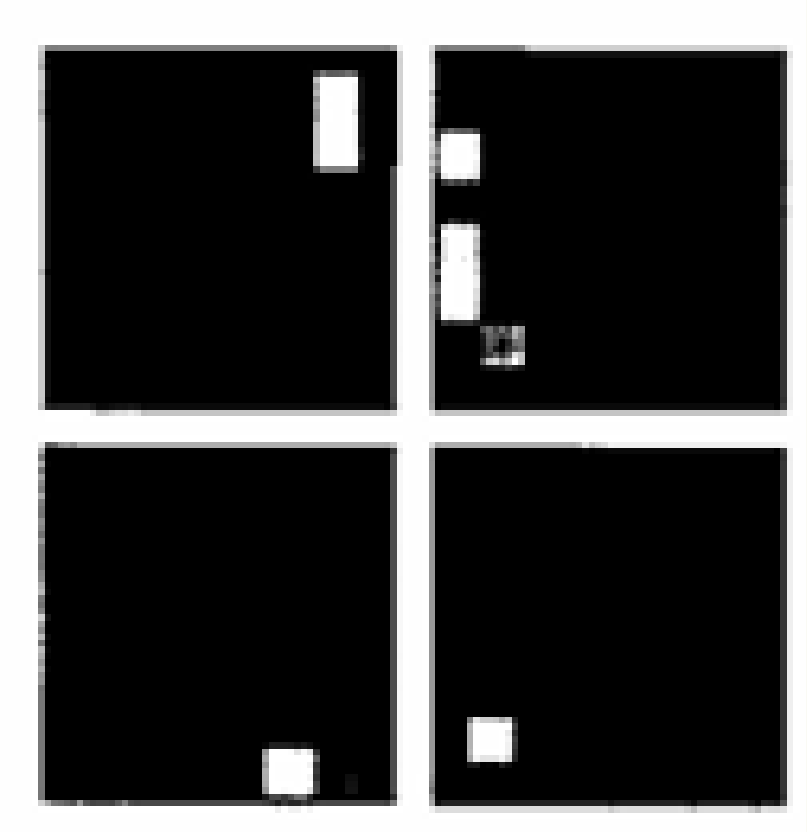
특징 맵 실험 결과 분석



- 마지막 컨벌루션 블록 → 여러 conv, pooling
  - 특징 맵 작아짐, 고수준의 특징 추출
- Conv_block3 : 18개 특징 맵
- 강아지 머리, 꼬리, 앞발, 뒷발 추출

01 | CNN

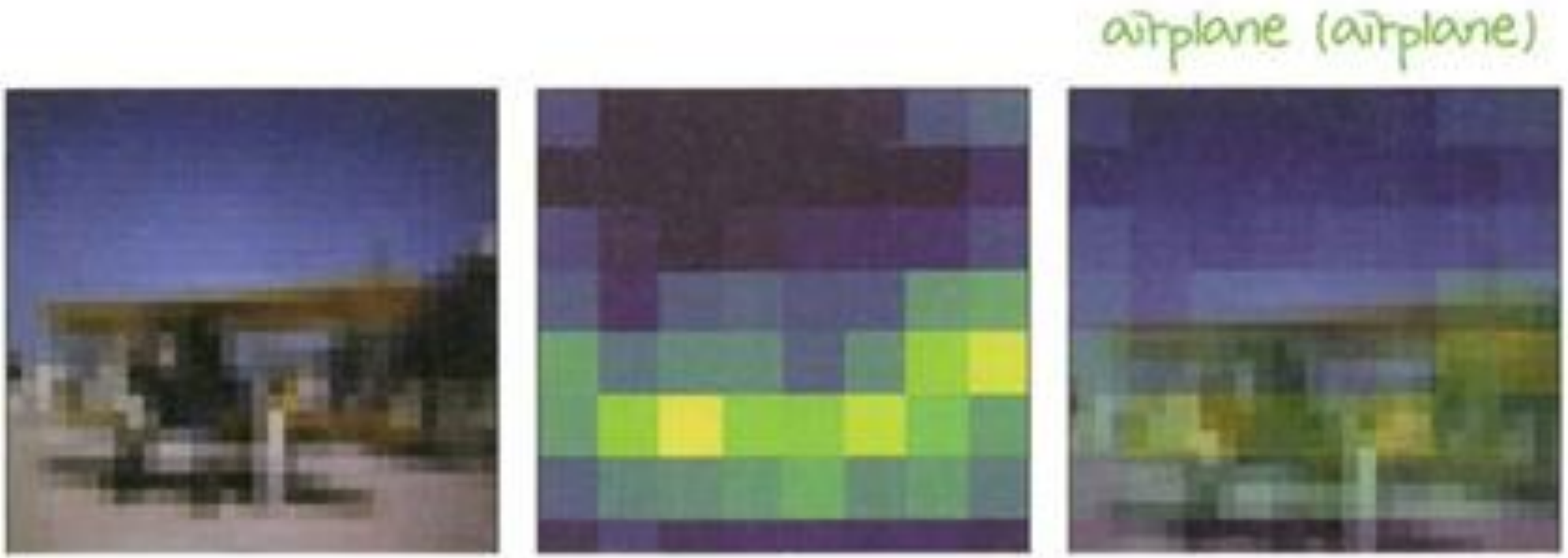
특징 맵 실험 결과 분석



- 노란색: 큰 값
- 파란색: 작은 값

01 | CNN

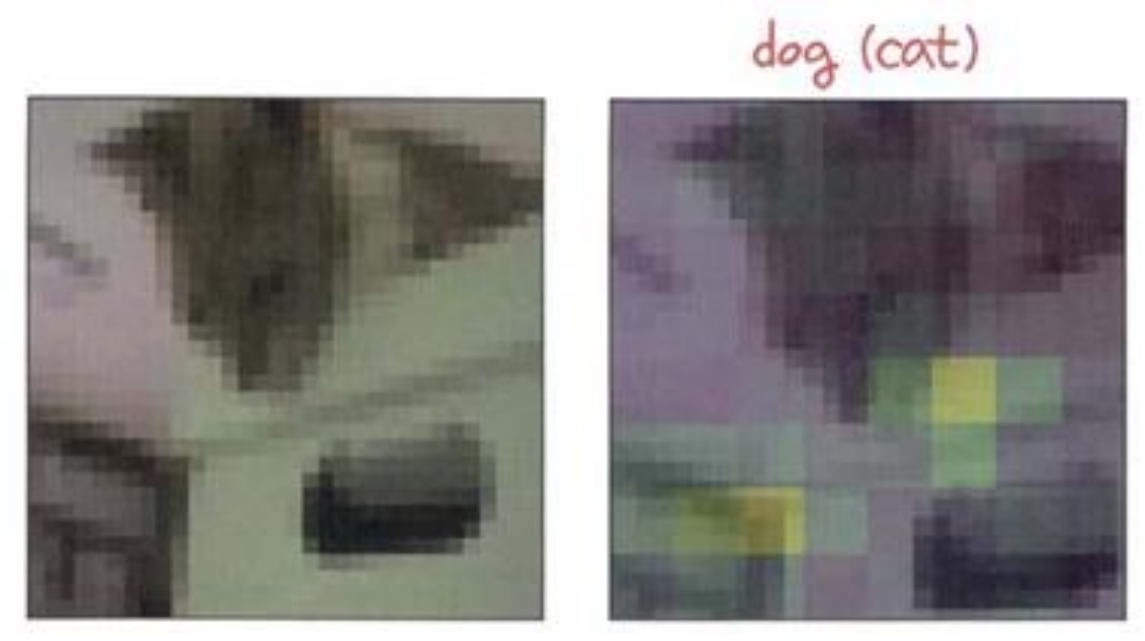
특징 맵 실험 결과 분석



- 노란색: 큰 값
- 파란색: 작은 값

01 | CNN

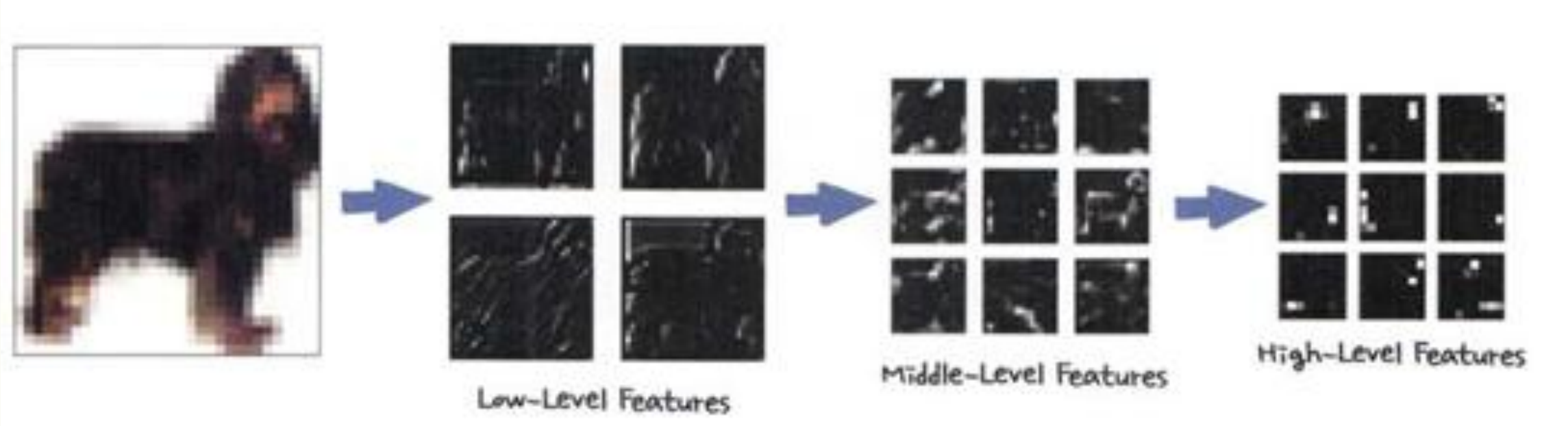
특징 맵 실험 결과 분석



- 잘못 분류
  - 입력: 박스 안에 들어있는 고양이
  - 출력: 강아지
- 오류 원인 해석: 박스에 과도하게 집중
- 고양이 머리 어느정도 주목 → 강아지로 분류 한 것으로 추측

01 | CNN

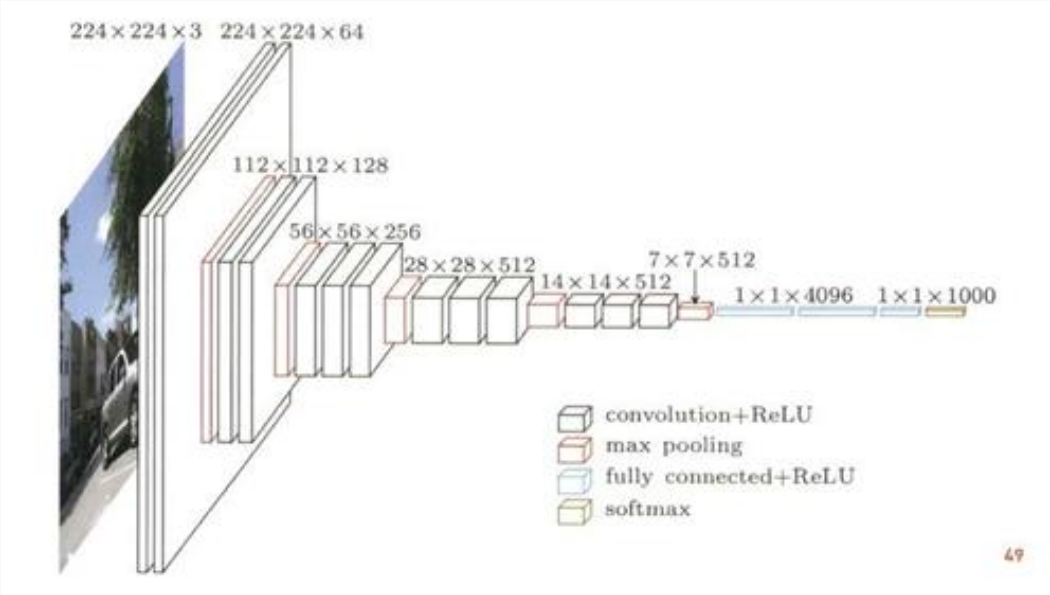
특징 맵 실험 결과 분석



- 컨벌루션 레이어가 깊어질 수록 더 복잡하고 추상적인 특징 포착
- 초기 층: 기본적인 특징
- 중간 층: 기본 특징 조합
- 마지막 층: 객체의 특정 부분, 전체적인 형태

02 | VGGNet

VGGNet 완벽 해부

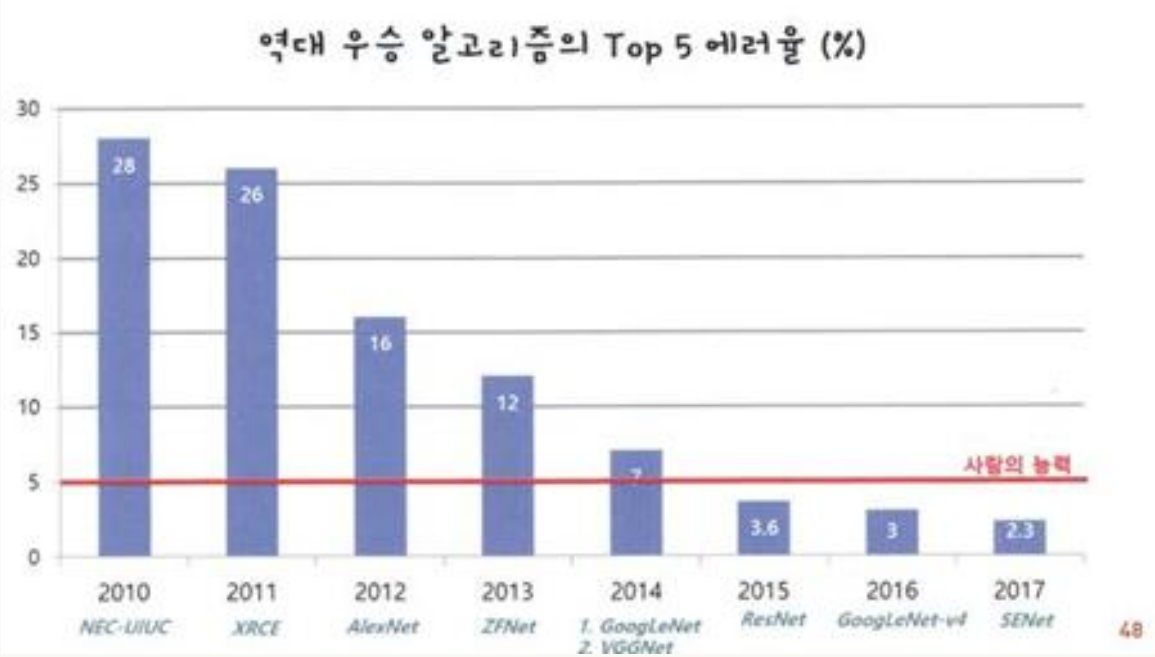


- VGGNet: Oxford 대학에서 개발한 모델
  - 2014년 ImageNet 챌린지 2위

02 | VGGNet

VGGNet 완벽 해부

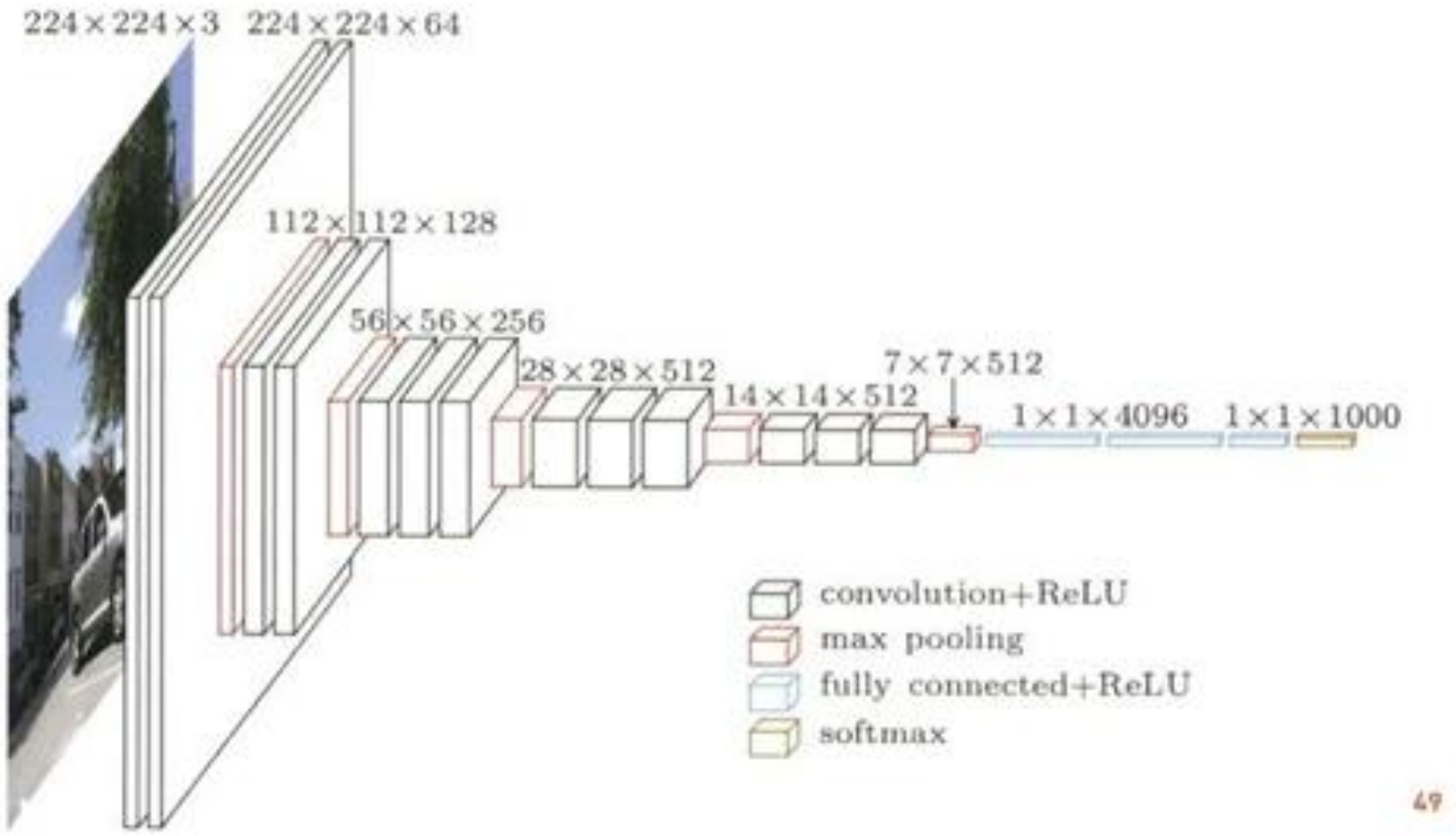
- IMAGENET: Training – 128만, Test – 10만, Validation – 5만
- 1000개 클래스 분류 대회



1. 모델은 각 이미지에 대해 1000개 클래스 각각의 확률 출력
2. 이 중 확률이 가장 높은 5개 클래스 선택
3. 실제 정답 클래스가 5개 중에 포함되어 있으면 정확한 예측으로 간주
4. 전체 테스트 셋으로 반복, 5개 안에 정답이 포함되지 않은 비율을 계산

02 | VGGNet

VGGNet 완벽 해부



02 | VGGNet

VGGNet 완벽 해부

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

1. 입력 : 244x244 크기의 RGB이미지
2. 첫 번째 컨볼루션 블록:
   - ‘conv3-64’: 커널 크기 3x3인 컨벌루션, 64개 필터 사용(64x3x3x3)
   - 결과 Shape: 64x244x244
   - 한번 더 ‘conv3-64’ 통과 (64x3x3x3)
   - 결과 Shape: 64x244x244
   - Max Pooling으로 크기 축소
   - 결과 Shape: 64x112x112 (채널 수는 유지)

02 | VGGNet

VGGNet 완벽 해부

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

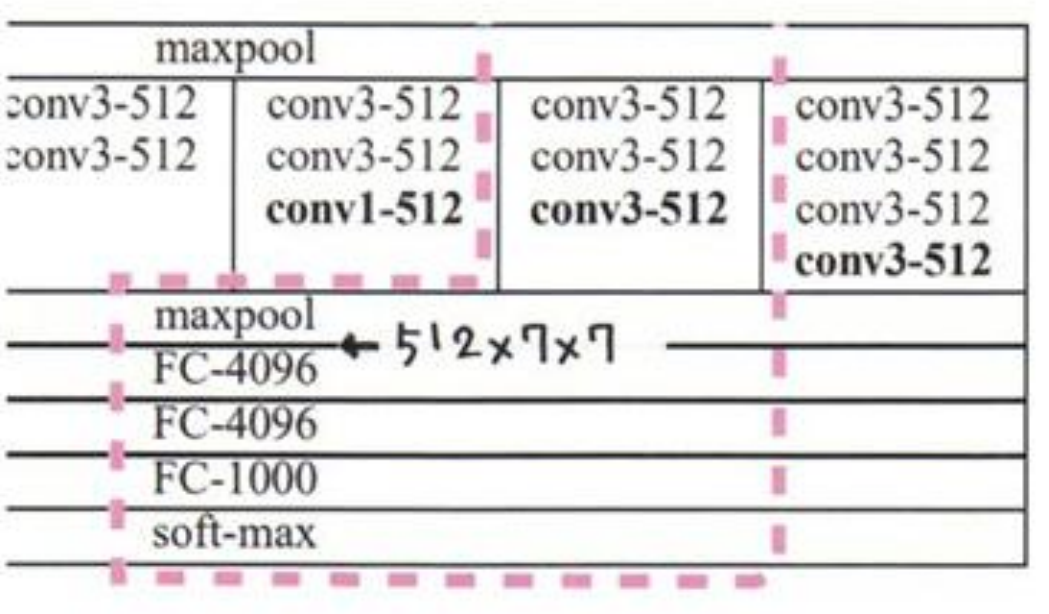
3. 두 번째 컨볼루션 블록:
   - ‘conv3-128’: 128개 필터 사용(128x64x3x3)
   - 결과 Shape:128x112x112
   - 한번 더 ‘conv3-128’ 통과 (128x128x3x3)
   - 결과 Shape:128x112x112
   - Max Pooling으로 크기 축소
   - 결과 Shape: 128x56x56(채널 수는 유지)

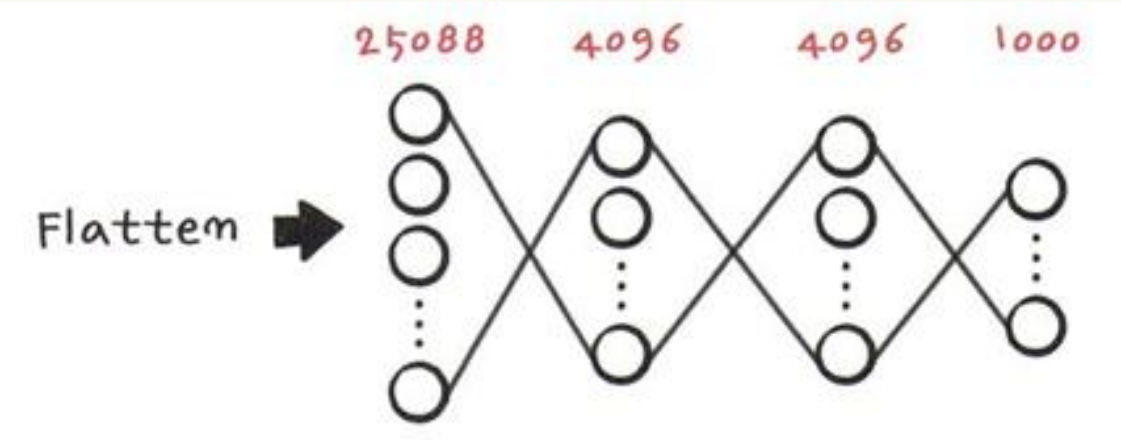
- 패턴 반복, 블록을 거치며 필터 수는 두 배씩 증가
  - 64 → 128 → 256 → 512

- 마지막 컨볼루션 블록 통과 후
  - 특징맵 : 512x7x7

02 | VGGNet

VGGNet 완벽 해부

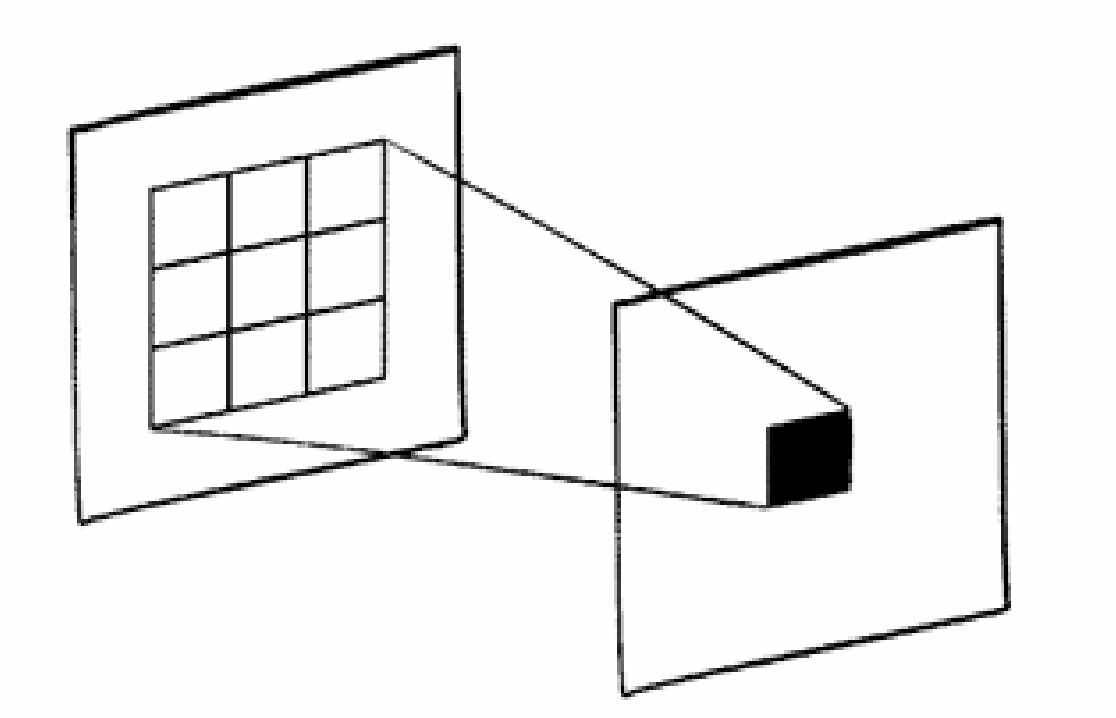




- Flatten: 512x7x7 크기의 특징 맵을 25088개의 1차원 백터로 변환
- MLP:
  - FC-4096: 25088개 노드를 4098개 노드로 연결
  - 다시 FC-4096: 4096개 노드를 유지
  - FC-1000: 최종적으로 1000개 노드로 축소

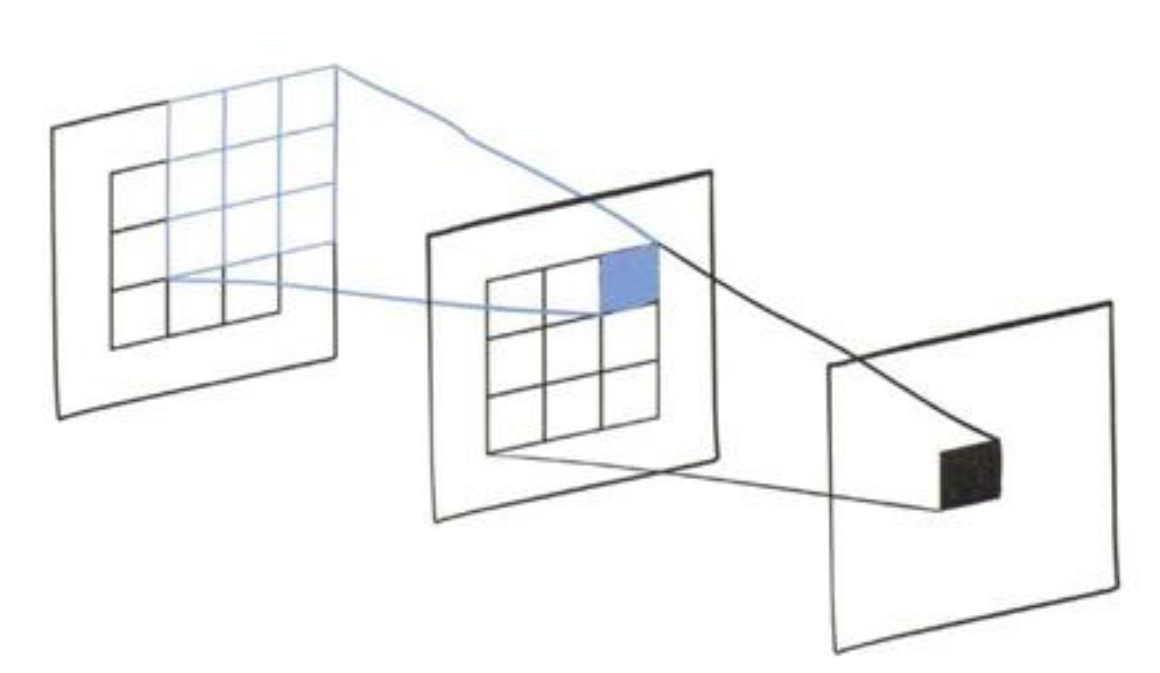
03 | Receptive Field 개념

- Receptive Field: 하나의 출력 픽셀 형성에 관여하는 입력 이미지 영역



1. 첫 번째 3x3 컨볼루션:
   - 출력의 한 픽셀은 입력의 3x3 영역을 직접 참조
   - 이때의 Receptive Field는 3x3

03 | Receptive Field 개념



2. 두 번째 3x3 컨볼루션:
   - 첫 번째 컨볼루션의 출력에 다시 3x3 컨볼루션을 적용
   - 이전 단계의 각 픽셀이 3x3 영역의 정보를 담고 있으므로, 두 번째 컨볼루션의 결과 픽셀은 더 넓은 영역의 정보를 포함하게 됨
   - 결과적으로 Receptive Field는 5x5로 확장

03 | Receptive Field 개념

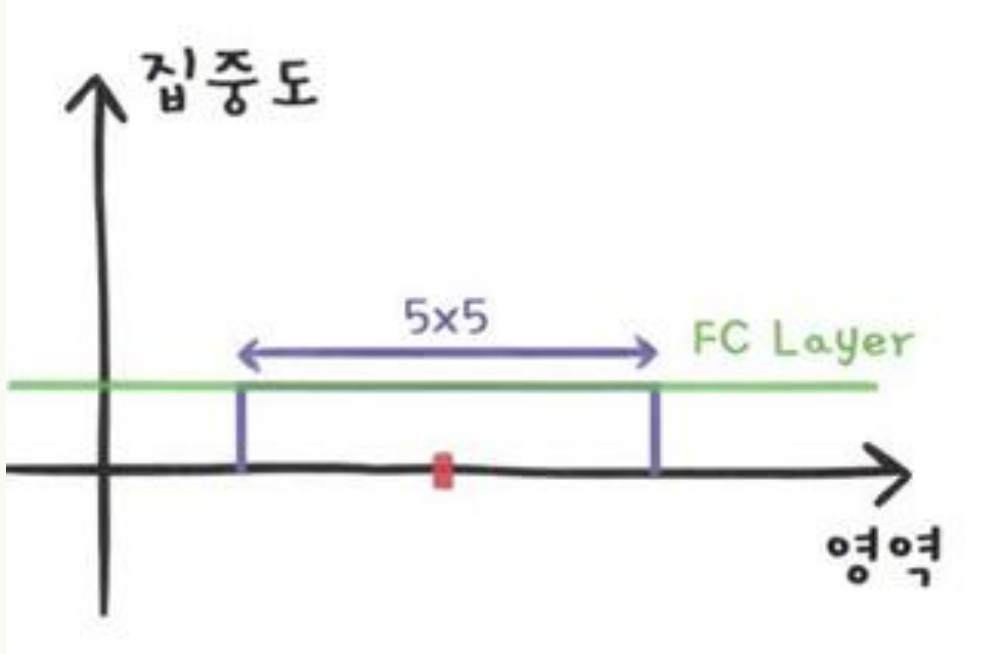
왜 처음부터 5x5 컨볼루션을 사용하지 않았는가?

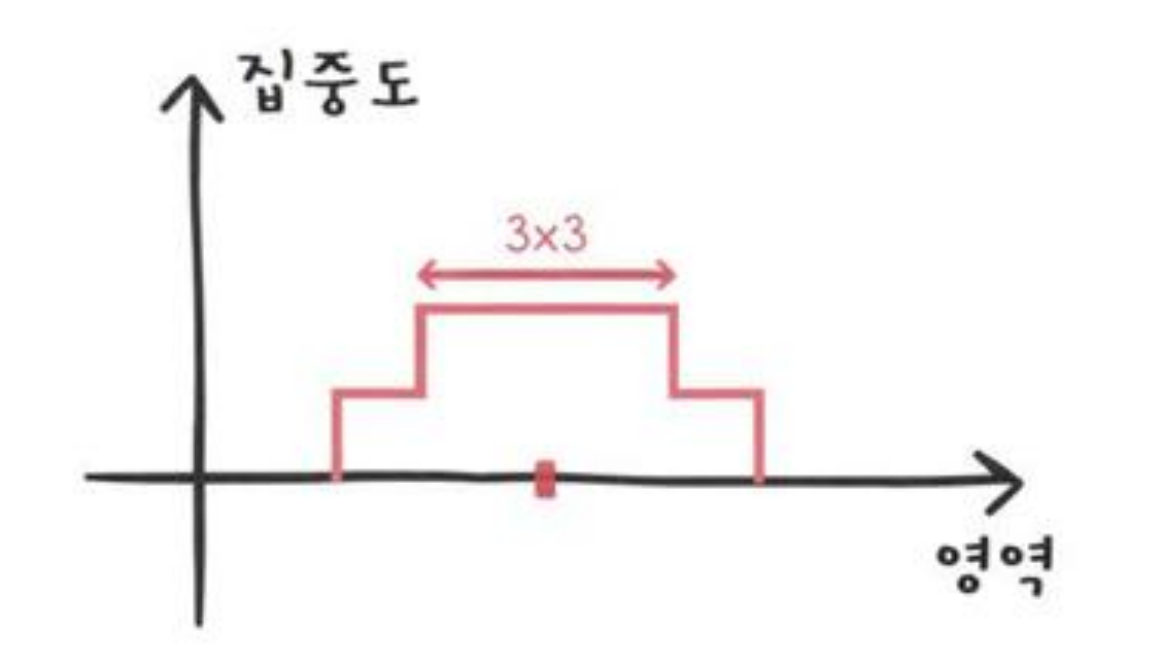
- 파라미터의 효율성: 3x3 컨볼루션 두번 파라미터 → 3x3x2 vs 5x5 파라미터 → 25
- 비선형성 증가: 활성화 함수를 더 많이 통과 --> 비선형성 증가 → 복잡한 패턴 학습

- 영역별 집중도 차이:

03 | Receptive Field 개념

- 영역별 집중도 차이:



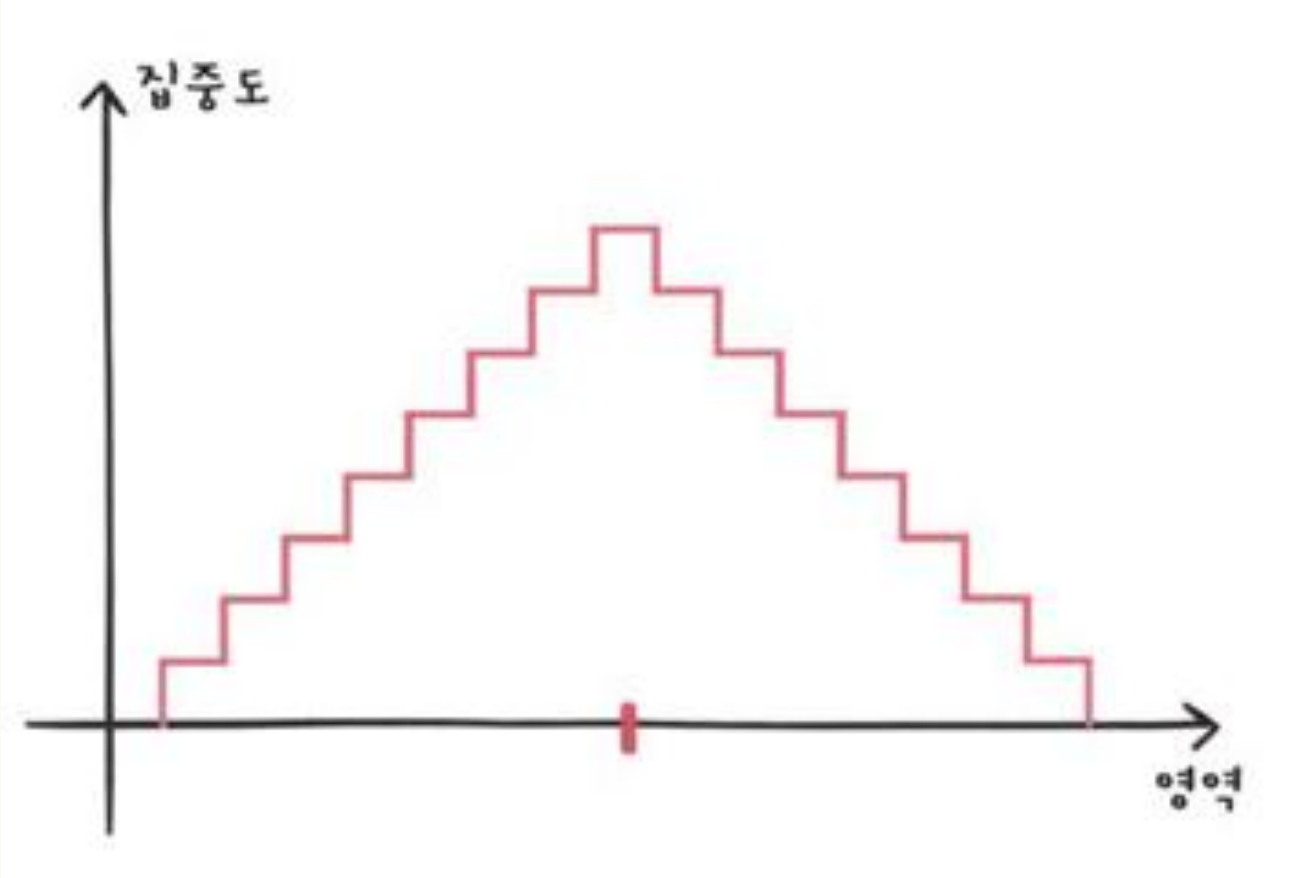


- 모든 픽셀 정보를 세세하게 활용 -->
- 공간적 구조 무시, Ovefitting

- 필요한 픽셀 정보들을 집중적으로 활용
- 가장자리 픽셀들은 간접적으로 관여

03 | Receptive Field

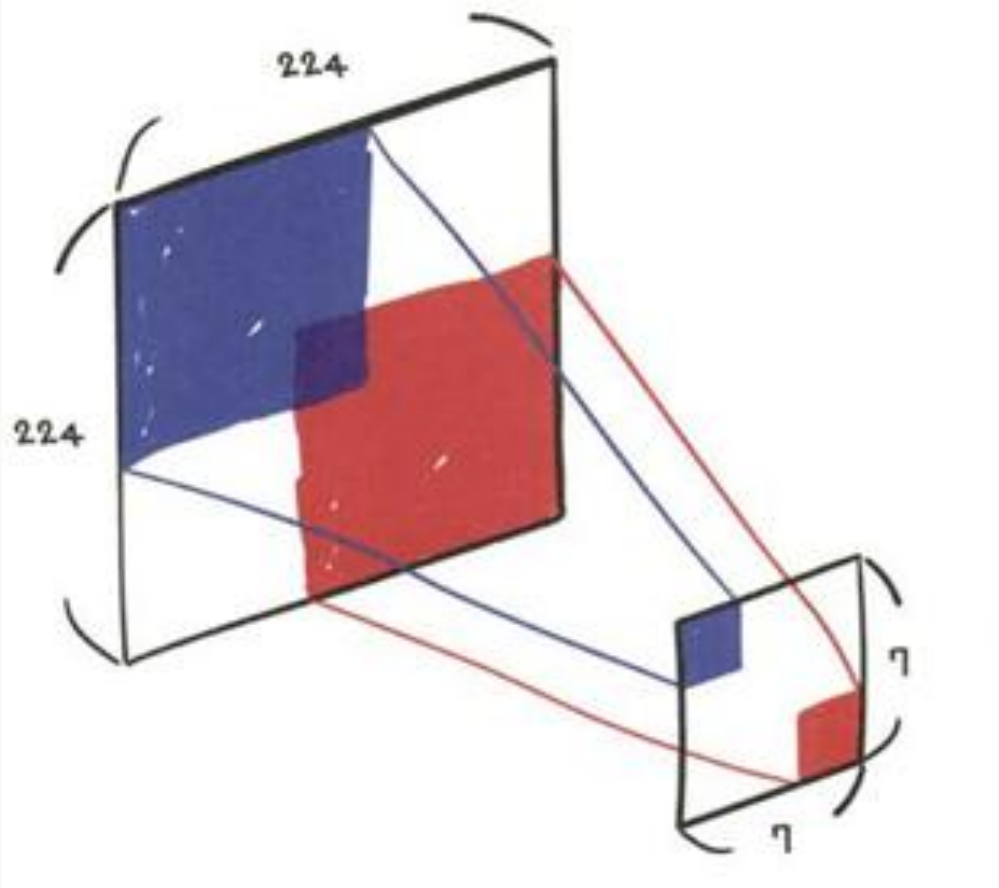
개념



- 담당하는 위치를 가장 집중해서 보면서도 주변 정보를 고려

04 | 고찰

고찰



- CNN의 강력한 특징 추출 능력은 다양한 분야에서 적용 가능
- CNN의 핵심 은 신경망 연결 구조를 효과적으로 설계한 것
  - 멀리 있는 정보도 활용하되, 여러 단계의 정제 과정을 거친 후에 활용

- 회전이나 크기 변화에 대한 불변성이 부족
  - 데이터 증강, 다양한 기법 연구 중

CNN, VGGNet, Receptive Field, 고찰

감사합니다